

Method of Neural Network Training with Integer Weights

Oleksandr Karpin
Fellow
Cypress Semiconductor
Lviv, Ukraine
oleksandr.karpin@cypress.com

Zinovii Liubun
Department of RadioPhysic and
ComputerTechnologies
Ivan Franko National University of Lviv
zinovijlyubun@gmail.com

Vasyl Mandziy
Prin Systems Engineer
Cypress Semiconductor
Lviv, Ukraine
vasyl.mandziy@cypress.com

Vasyl Rabyk
Department of RadioPhysic and
ComputerTechnologies
Ivan Franko National University of Lviv
rabyk@ukr.net

Abstract— This paper presents the method of training the feedforward neuronal network with integer weights and integer input and output signals. Neural networks of this type are better suited for embedded systems and hardware implementation than neural networks with floating weights. Also, the new method allows training neural networks on embedded devices with limited resources.

This algorithm has been developed taking into account that the resulting integer weights require less memory for storage, and arithmetic operations over them are easier for hardware implementation.

A simple activation function is proposed that works with integer data and limits weights and bias in a certain range during neural network training. The simulation results show that the new training method gives a similar result while training with floating-point arithmetic for XOR operation.

To validate the robustness of the new algorithm to train neural networks with integer arithmetic, a prototype was created based on the CYAT817 microcontroller with the Cortex-M0+ core.

Keywords—neural network, integer weights

I. INTRODUCTION

Advancing of neural network technologies into solving various processing information tasks boosted the need to implement neural networks on digital devices with limited resources both in memory and in speed. This in turn requires new approaches to the implementation of neural networks, in particular, using neural networks with integer weights and activation functions that work with the integer argument. In addition, new approaches are needed to implement algorithms for training such neural networks. Therefore, this problem is given considerable attention [1-3].

To study the peculiarities of the implementation of the integer perceptron network, the simplest double-layer feedforward neural network was chosen – Fig. 1.

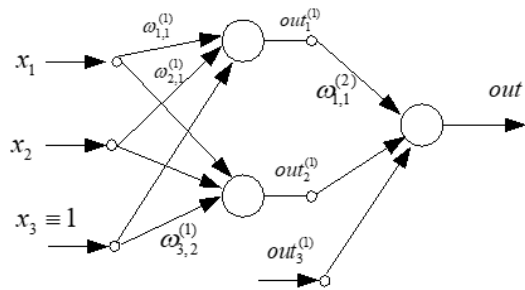


Fig. 1. Double-layer feedforward neural network

The power function given by Eq. (1) is used as an activation function:

$$Out = \begin{cases} \text{int} \left(\frac{3 \cdot M_y \cdot S}{2 \cdot M_x} \right) - \text{int} \left(\frac{M_y \cdot S^3}{2 \cdot M_x^3} \right), & \text{abs}(S) < M_x \\ M_y \cdot \text{sign}(S), & \text{abs}(S) \geq M_x \end{cases} \quad (1)$$

Value $S = \overrightarrow{Out^{(1)}} \overrightarrow{w^{(2)}}$ for the outcome neuron and $S = \vec{x}^{(1)} \vec{w}_1^{(1)}$ for hidden layer neurons.

The activation function derivative is determined by Eq. (2):

$$Out = \begin{cases} \frac{3 \cdot M_y}{2 \cdot M_x} \cdot \left(1 - \left(\frac{S}{M_x} \right)^2 \right), & \text{abs}(S) < M_x \\ 0, & \text{abs}(S) \geq M_x \end{cases} \quad (2)$$

The graph of the function itself and its derivative is in Fig. 2 and Fig. 3:

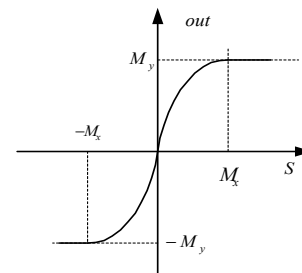


Fig. 2. Power activation function

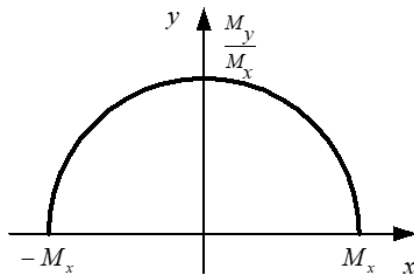


Fig. 3. Derivative of power activation function

In the work [1], it is proposed to use the representation of standard activation functions, for example, a hyperbolic tangent in the form of a table of integer values. Such a representation is quite acceptable in the implementation of the neural network as a program where each network neuron's outputs are calculated consistently, therefore it is possible to use the function table representation. However, the important property of the calculation parallelism inherent in the neural networks structure is not used. Therefore, hardware implementation of the neural network may do a better use of the proposed cubic function.

In some algorithms for training with integer data, only the sign of the activation function derivative is taken into account. We think that the best training outcomes can be obtained taking into account not only the derivative sign but also its value which gradually decreases when S approaches the $\pm M_x$ value (Fig. 3).

The network was trained using the modified version of our RProp (Resilient Propagation) [4]. This algorithm advantage is its easy adaptation for work with integer data where weight correction values for each training pair must be determined as an integer. The problem is that the correction value depends on the value of the activation function derivative, and for most activation functions, it is smaller than 1, including our selected activation function. Therefore, instead of the derivative given Eq. (2), a function is used that preserves the characteristic behavior of the derivative and at the same time has integer values:

$$Out = \begin{cases} M_y - int\left(\frac{M_y \cdot S^2}{M_x^2}\right), & abs(S) < M_x \\ 0, & abs(S) \geq M_x \end{cases} \quad (3)$$

As in the classical algorithm, the minimum weights correction value Δ_{min} is given, the maximum correction value Δ_{max} , the value by which the weight correction increases Δ_+ , and the value of the decrease Δ_- . Obviously, all these values are integers, for example, Table I shows the value Δ_0 that corresponds to the initial correction value for all weights.

TABLE I. WEIGHTS CORRECTIONS VALUES

Δ_{min}	1
Δ_{max}	12
Δ_+	1
Δ_-	2
Δ_0	6

Initial and maximum weight correction values, as well as values Δ_+ and Δ_- for each specific task, may be different. Obviously, always $\Delta_- > \Delta_+$.

To determine weight corrections values in the following training steps, as well as for a floating case, the back-propagation algorithm is used together with the method of the fastest descent:

for the i -th weight of the output neuron –

$$\delta_i = \frac{\partial E}{\partial w_i^{(2)}} = (Out - T) \frac{\partial Out}{\partial S} Out_i^{(1)}$$
, where T – the target outcome

for the i, j -th weight of a j -th hidden layer neuron –

$$\delta_{ji}^{(1)} = \frac{\partial E}{\partial w_{ji}^{(1)}} = (Out - T) \frac{\partial Out}{\partial S} w_i^{(2)} \frac{\partial Out_i^{(1)}}{\partial S_i^{(1)}} x_j$$

For Batch mode, the values obtained for weights correction are summed, and weights correction values are determined on the basis of them and their value in the previous step ($t-1$). The following algorithm of choosing weights correction values is used:

$$\begin{aligned} & \text{if } \left(\text{sign} \left(\sum \delta_{ji}(t-1) \right) \cdot \text{sign} \left(\sum \delta_{ji}(t) \right) > 0 \right) \text{ then} \\ & \quad \Delta_{ji}(t) = \Delta_{ji}(t-1) + \Delta_+ \\ & \quad \text{if } (\Delta_{ji}(t) > \Delta_{max}) \text{ then } \Delta_{ji}(t) = \Delta_{max} \\ & \quad w_{ji}(t) = w_{ji}(t-1) - \text{sign}(\delta_{ji}(t)) \cdot \Delta_{ji}(t) \\ & \text{else if } \left(\text{sign} \left(\sum \delta_{ji}(t-1) \right) \cdot \text{sign} \left(\sum \delta_{ji}(t) \right) < 0 \right) \text{ then} \\ & \quad \Delta_{ji}(t) = \Delta_{ji}(t-1) - \Delta_- \\ & \quad \text{if } (\Delta_{ji}(t) < \Delta_{min}) \text{ then } \Delta_{ji}(t) = \Delta_{min} \\ & \quad w_{ji}(t+1) = w_{ji}(t) - \Delta_{ji}(t-1) \\ & \quad \sum \delta_{ji}(t) = 0 \\ & \text{else if } \left(\text{sign} \left(\sum \delta_{ji}(t-1) \right) \cdot \text{sign} \left(\sum \delta_{ji}(t) \right) = 0 \right) \text{ then} \\ & \quad w_{ji}(t) = w_{ji}(t-1) - \text{sign}(\delta_{ji}(t)) \cdot \Delta_{ji}(t-1) \end{aligned} \quad (4)$$

Ensuring weights values and accordingly S values be limited in the training process is an important issue when working with integer weight values and the limited size of variables. The operation of the network becomes unpredictable under an overflow.

The localization property of neural networks is that each network neuron is a local device that receives data on the input, processes it, and transmits to next neurons. This fact is enough to prove that under certain conditions in the training process, neural network weights increase is limited.

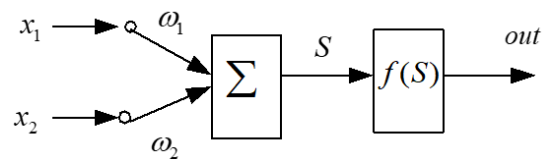


Fig. 4. Neuron with two-dimensional input

Suppose we have a neuron to which input a two-dimensional signal $\vec{x} = (x_1, x_2)$ is sent (Fig. 4). Unlimited weight increase is possible only if the value $S = x_1 \cdot w_1 + x_2 \cdot w_2$ remains limited, but the terms have different signs.

Suppose,

$$x_1, x_2 > 0 \quad (5)$$

Then, for the unlimited increase:

$$\text{sign}(w_1) \neq \text{sign}(w_2) \quad (6)$$

Suppose, condition (6) is fulfilled, and

$$\begin{cases} w_1 > 0 \\ w_2 < 0 \end{cases} \quad (7)$$

In addition, for an unlimited weight increase, the following condition must be fulfilled for the weight correction values:

$$\begin{cases} \Delta w_1 > 0 \\ \Delta w_2 < 0 \end{cases} \quad (8)$$

Therefore, in the training process, the positive weight increases and the negative decreases. Suppose, the least descent method is used to correct the weights, then the weights correction values are determined as follows:

$$\begin{aligned} \Delta w_i &= \eta \frac{\partial E}{\partial w_i} = \eta (Out - T) \cdot \frac{\partial Out}{\partial S} \cdot \frac{\partial S}{\partial w_i} = \\ &= \eta (Out - T) \cdot \frac{\partial Out}{\partial S} \cdot x_i \end{aligned} \quad (9)$$

Taking into account that the values of η , $\frac{\partial Out}{\partial S}$ and x_i are always positive, the growth sign is determined only by the difference and have the same sign for both weights, hence condition (8) is not fulfilled. Therefore, weights increase is limited.

Weight values change may vary. This leads to an increase or decrease of S , which in turn leads to the activation function derivative reaching zero as well as the value of weights correction. Therefore, unlimited weight increase is impossible.

A similar result can be obtained for the case when:

$$x_1, x_2 < 0 \quad (10)$$

Then, for an unlimited increase:

$$\text{sign}(w_1) = \text{sign}(w_2) \quad (11)$$

Suppose $\text{sign}(w_1) > 0$. For an unlimited increase of weights:

$$\begin{cases} \Delta w_1 > 0 \\ \Delta w_2 > 0 \end{cases} \quad (12)$$

As in the previous case, taking into account (9), we obtain that, under an unlimited increase, the weights correction values have different signs, which contradicts condition (12).

In practice, we can propose the following algorithm for controlling the maximum weights values. Under weights

increase, the overflow is caused by an unacceptably high value at the exit of an individual summator. For a specific problem, having the maximum values of the input vector components, the vector's dimension and maximum value at the neuron output (this value plays the role of the input vector for the next layer of the neural network), we can estimate the maximum permissible value of the module of neural network weight in the training process w_{max} . Then, the maximum number of training periods which guarantees not leading to overflowing, can be estimated as:

$$N_{max} \leq \frac{w_{max} - w_{max}^0}{\Delta_{max}}, \quad \text{where } w_{max}^0 \text{ the maximum initial weight value.}$$

So, you can train the network during N_{max} training periods. If during the N_{max} training periods, the desired training result is not achieved, then you can continue training after analyzing the maximum weight value in the last iteration or forcing down the weight value, for example, according to the algorithm described in [5].

II. RESULTS

The training algorithm described above, with the selected activation function, was tested on the classification task for the XOR function, Table II.

TABLE II. XOR FUNCTION

x_1	x_2	x_3	T
100	100	100	100
-100	100	100	-100
100	-100	100	-100
-100	-100	100	100

In spite of the simplicity, this task is characterized by the presence of local minima which we often find when training a neural network using the gradient methods. Therefore, the result of training essentially depends on the initial choice of weights. Integer initial values of the weights were generated using the random function in the range $[-100, +100]$. Below you will find that in this task there are two global minima whose training error is practically zero but they correspond to fundamentally different sets of neural network weights.

To study the training and functioning of the integer neural network, a emulator of the neural network in Python using the NumPy matrix operations module and the matplotlib library was used to visualize the training and operation of the neural network.

Figure 5 shows the results of testing a trained network with an absolute error value of less than two. In Fig. 5, the dot indicates the area of the input data which at the output of the trained network gives a value greater than 90, and the plus - the value at the output less than 10. In Fig. 6, the same result is shown in the three-dimensional area where the values at the output of the trained network are placed vertically. The values of the trained network weights are shown in Table III.

TABLE III. ANN WEIGHTS FOR FIRST GLOBAL MINIMUM

Input Layer Weights		Output Neuron Weights
-508	595	-342
-545	638	-341
308	477	409

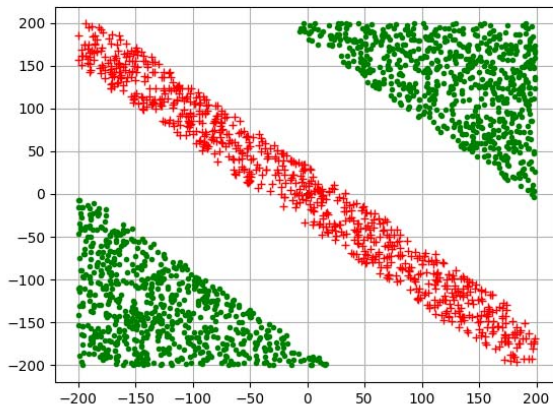


Fig. 5. Dependence between ANN output and inputs after reaching first global minimum

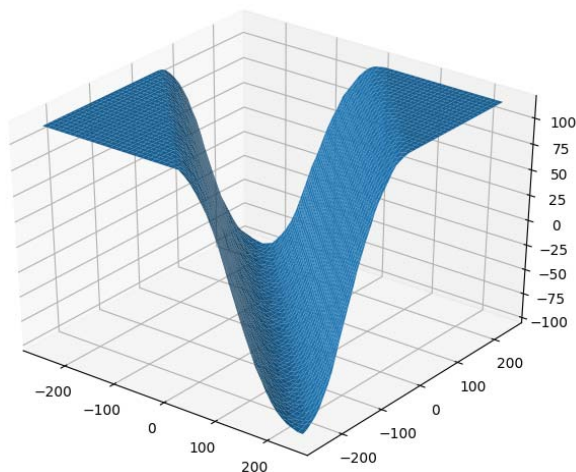


Fig. 6. Surface of trained network output under reaching first global minimum

When training begins with other randomly selected weight values, the network may come to another global minimum. From our task point of view, we obtained the solution with the same accuracy but obtained another set of network weights and accordingly we have different distribution of the input data area, depending on the value at the trained network output as shown in Fig. 7, and in the representation as a surface of distribution in Fig. 8.

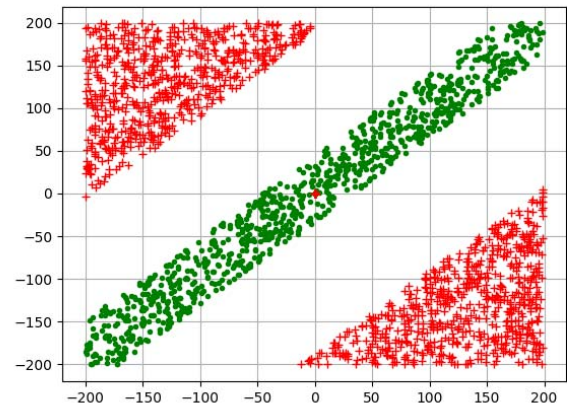


Fig. 7. Dependence between ANN output and inputs after reaching second global minimum

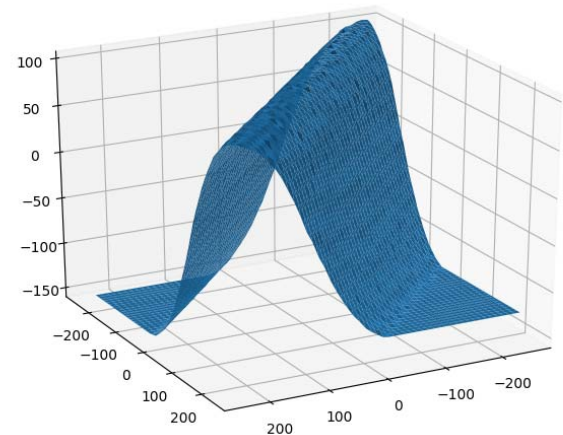


Fig. 8. Surface of trained network output under reaching second global minimum

The weights values of the trained network when reaching second global minimum, Table IV.

TABLE IV. ANN WEIGHTS FOR SECOND GLOBAL MINIMUM

Input Layer Weights		Output Neuron Weights
-494	-474	-410
510	-487	-376
-485	-286	-517

As to the optimization problem in the multidimensional space of the network weights values, there are two global minima that give an error practically equal to zero and for Fig. 5-8, they correspond to different behavior in the input data space.

III. FW IMPLEMENTATION

To validate the robustness of the new algorithm to train neural networks with integer arithmetic, a prototype was created based on the CYAT817 microcontroller with the Cortex-M0+ core.

Table V shows the memory consumption and processing time for the ANN learning with integer weights on chip for the XOR operation.

TABLE V. MEMORY AND PROCESSING TIME FOR ANN LERNING WITH INTEGER WEIGHTS ON CYAT817 FOR XOR OPERATION

Memory, bytes	976
Processing time for ANN learning	$0.1 \text{ ms} * 1000 \text{ epoch} = 0.1 \text{ s}$

IV. CONCLUSIONS

The neuroemulator of a dual-layer neural network written in Python showed that an integer network can function successfully using both the tangential and the proposed by us cubic activation function. The cubic function is more acceptable when a neural network is implemented where its parallelism is used, then computation is performed in all layer neurons simultaneously and there is no need to use the activation function specified in the table. In this case, the simplicity of calculating the cubic function has its advantages as with the hardware implementation of the neural network.

The proposed modification of the RProp algorithm for training neural networks with integer configuration showed not worse results compare to the floating-point weights values.

The proposed method of controlling the weight values makes it possible to guarantee their limitations in the training process which was confirmed by numerical experiments.

REFERENCES

- [1] Lai Liangzhen, Suda Naveen, Chandra Vikas, "Efficient neural network kernels for arm cortex-m cpus-," arXiv preprint, arXiv:1801.06601, 2018 - arxiv.org.
- [2] Yi Yan, Hangping Zhang, Bin Zhou, "A New Learning Algorithm for Neural Networks with Integer Weights and Quantized Non-linear Activation Functions," in IFIP AI 2008: Artificial Intelligence in Theory and Practice II, pp 427-431.
- [3] V.P. Plagianakos, M.N. Vrahatis, "Training Neural Networks with Threshold Activation Functions and constrained Integer Weights. Neural Networks, " in Proceedings of the IEEE-INNS-ENNS International Joint Conference, IJCNN 2000, 2000, vol. 5, p. 161.
- [4] Igel Christian and Hüsken Michael, "Empirical evaluation of the improved Rprop learning algorithms, " Neurocomputing, vol. 50, pp. 105-123, September 2002.
- [5] H. D. Wasserman. Neural computing: theory and practice. Van Nostrand Reinhold Co. New York, NY, USA ©1989 ISBN:0-442-20743-3.